

Governance-Gated Credit-Risk Benchmarking: Empirical-Bayes Segment Shrinkage on the UCI Default of Credit Card Clients Dataset

Aryan Singh

github.com/asynced24/financial-ai-risk-governance-framework (v1.0.0)

August 2026

Abstract

Model selection and model approval ask different questions of a credit-risk model, and they are usually answered by different artefacts at different times. This report describes a pipeline that answers both in one run. Three credit-default models are benchmarked on the UCI *Default of Credit Card Clients* dataset and each is then put through five governance gates — fairness, drift, calibration, uncertainty, and a Bayesian segment-stability check — whose thresholds are declared in configuration and validated at startup. Segment-level default rates, the unit most governance questions are posed in, are estimated with an empirical-Bayes Beta-Binomial model rather than taken raw, so that cells holding a handful of borrowers get a defensible rate and an honest interval instead of noise. On the committed offline sample all three models fail the combined verdict. The binding finding is an equalized-odds gap of 0.1617 on **sex** for logistic regression, against a fail limit of 0.10 — produced by a model that never received **sex** as a feature. A second finding is that governance verdicts reproduce across platforms while the benchmark ranking does not: XGBoost’s histogram tree builder moves ROC-AUC by more than the margin separating the top two models.

1 Problem statement

A credit-scoring model has to pass model-risk review before it can inform a lending decision, and review asks a different set of questions from the ones model selection asks. Selection asks which model ranks borrowers best. Review asks whether a predicted probability means what it claims, whether error rates are comparable across demographic groups, whether the input distribution has moved since training, and whether an individual adverse decision can be explained to the person who received it.

Those two sets of questions are normally answered by separate artefacts: a training script produces the discrimination metrics, and a document assembled weeks later addresses the review questions, often from summary statistics rather than from a check that runs. The separation matters because the two can disagree, and a model with the best ROC-AUC in a benchmark can still be one that should not be deployed.

This framework runs both in the same pipeline. Every threshold lives in `config.yaml`, is validated by pydantic at startup, and yields a pass/warn/fail verdict on every run; the worst gate becomes the model’s verdict. Selection and approval are separate outputs of a single execution.

A second concern is segment granularity. Governance questions are usually posed per demographic segment, and a segment default rate computed directly is dominated by sampling noise once the cell is small: in the run reported here, one segment holds six borrowers and zero defaults, and its raw default rate of 0.000 is not a usable input to any provisioning decision. Section 3 describes the empirical-Bayes model used in place of raw rates.

2 Architecture

Figure 1 shows the pipeline. The loader fetches UCI dataset 350, caches it, and can instead read a committed 5,000-row class-stratified sample so that the whole pipeline runs with no network access. The processor engineers 28 behavioural features from the six months of billing and repayment history, giving 49 predictors, and splits the data. It also separates *audit columns* from the feature matrix: **sex** and **marriage** are withheld from training entirely as prohibited bases and retained only so the fairness gate can measure against them.

Two components then feed the gates. The benchmark suite fits the three models and produces test-set predictions and metrics. Independently, the shrinkage model fits a population prior and per-segment posteriors on the training split. The five gates consume both, and each returns a pass/warn/fail verdict with the metric and threshold that produced it. Explainability and reporting run after the gates, and every parameter and metric is logged to MLflow or, when MLflow is disabled, to a local JSON run log.

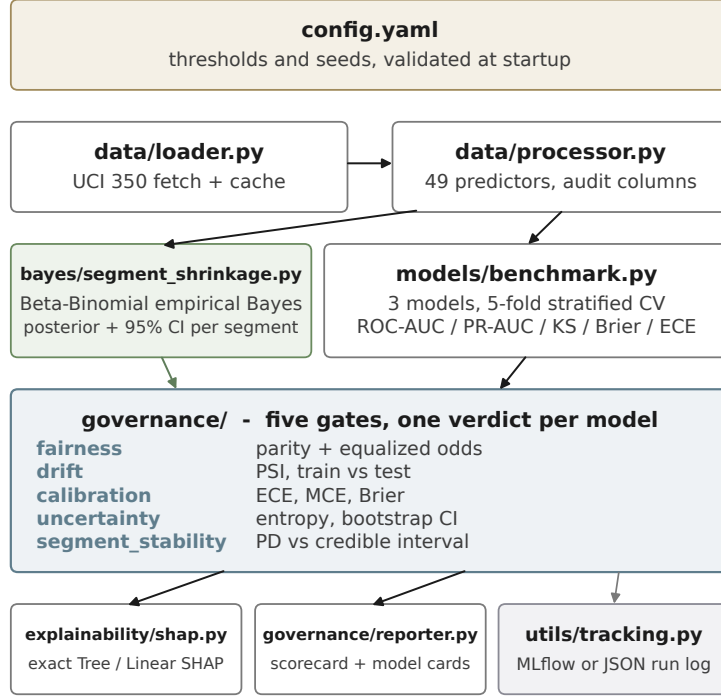


Figure 1: Pipeline stages. Thresholds, seeds and the model list are declared once in `config.yaml` and read by every stage below it. The benchmark suite and the empirical-Bayes shrinkage model are independent inputs to the gate layer: the first supplies predicted probabilities, the second the segment posteriors the stability gate compares them against.

3 Empirical-Bayes segment shrinkage

Segments are defined as the cross of education level and age band, giving 19 cells on this dataset. Rather than take each cell’s observed default rate, the framework treats the counts hierarchically: each segment has its own latent default rate, and those rates are themselves draws from one population-level distribution. Because the Beta is conjugate to the Binomial, every segment’s posterior is closed form, and its mean is automatically a precision-weighted blend of the segment’s own rate and the population rate. The weight is not a tuned constant — it falls out of the posterior algebra, so a thin cell shrinks hard toward the population rate and a well-populated cell barely moves. For segment i with n_i borrowers and k_i defaults:

$$\theta_i \sim \text{Beta}(\alpha, \beta), \quad k_i \sim \text{Binomial}(n_i, \theta_i), \quad (1)$$

$$\theta_i \mid k_i, n_i \sim \text{Beta}(\alpha + k_i, \beta + n_i - k_i), \quad (2)$$

$$\mathbb{E}[\theta_i \mid k_i, n_i] = w_i \frac{k_i}{n_i} + (1 - w_i) \frac{\alpha}{\alpha + \beta}, \quad w_i = \frac{n_i}{n_i + \alpha + \beta}. \quad (3)$$

The 95% credible interval is read off the same posterior as the 2.5th and 97.5th percentiles of $\text{Beta}(\alpha + k_i, \beta + n_i - k_i)$.

The prior (α, β) is estimated by method of moments across segments, with one correction that matters. The observed spread of the empirical rates $p_i = k_i/n_i$ overstates the spread of the underlying θ_i , because each p_i carries its own binomial sampling noise:

$$\text{Var}(p_i) = \text{Var}(\theta_i) + \mathbb{E} \left[\frac{\theta(1 - \theta)}{n_i} \right]. \quad (4)$$

Subtracting the sampling component gives $\text{Var}(\theta) = \text{Var}(p_i) - \overline{\mu(1 - \mu)/n_i}$, floored at a small positive constant, and the Beta concentration then follows from the identity $\text{Var} = \mu(1 - \mu)/(\alpha + \beta + 1)$. Using $\text{Var}(p_i)$ directly instead yields a prior that is too diffuse and therefore under-shrinks. Only segments with $n_i \geq 30$ contribute to the prior fit; every segment receives a posterior regardless.

On the run reported here, 12 of the 19 segments cleared the prior-fitting floor. The pooled rate is $\mu = 0.2230$, the observed variance of segment rates is 1.4581×10^{-3} and the estimated binomial sampling variance is 1.0243×10^{-3} , leaving $\text{Var}(\theta) = 4.3375 \times 10^{-4}$. That gives a concentration of $\mu(1 - \mu)/\text{Var}(\theta) - 1 = 398.49$ and a prior of $\text{Beta}(88.87, 309.62)$ — in other words, a prior worth about 398 equivalent borrowers.

Figure 2 shows the effect. The seven segments left of the boundary sit below the prior-fitting floor, and their raw rates scatter from 0.000 to 0.333 while their posteriors all sit within a few points of the population rate with wide intervals. The thinnest, `other_unknown | 50-60`, has $n = 6$ and $k = 0$: its posterior is $\text{Beta}(88.87, 315.62)$, mean 0.220, interval $[0.181, 0.261]$, and by Equation 3 the weight on its own data is $6/(6 + 398.49) = 0.01$. To the right, posteriors track their raw rates closely and the intervals tighten. Across all 19 segments the median interval width is 0.0732 and the median weight on own data is 0.212.

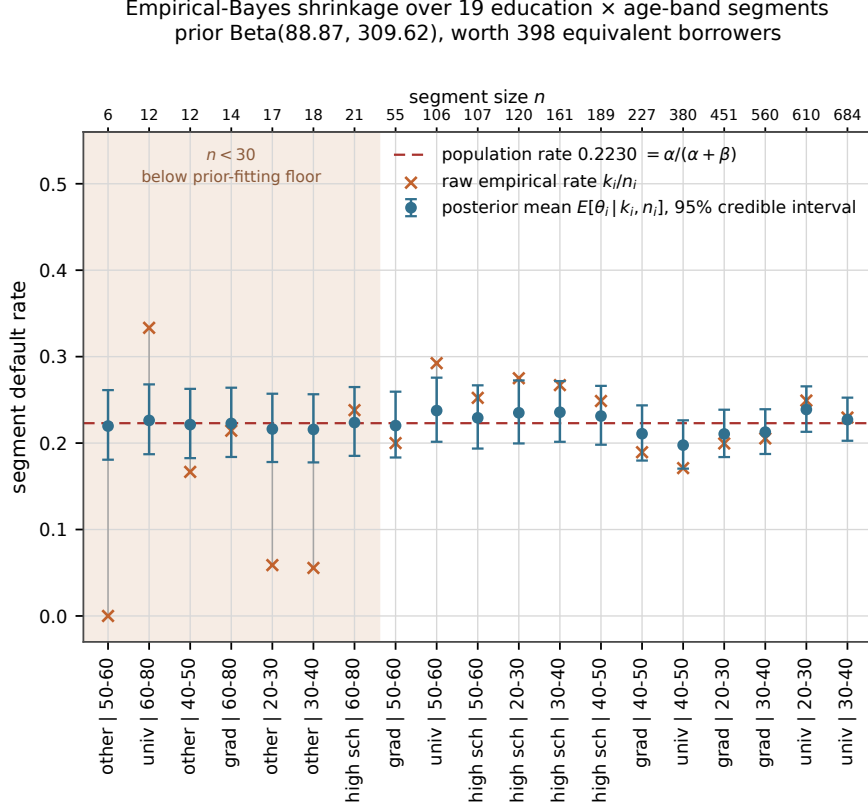


Figure 2: Raw versus shrunk segment default rates, 19 education $\times$ age-band segments ordered by segment size (sizes on the top axis). Crosses are the raw empirical rates k_i/n_i ; circles are posterior means with 95% credible intervals; the grey stems show how far shrinkage moved each estimate. Thin segments (shaded, $n < 30$) collapse toward the population rate with wide intervals; well-populated segments barely move. Produced by fitting the model on the training split of the committed sample.

The segment-stability gate then compares each model’s mean predicted probability of default per segment against these intervals, and flags any segment where the model prices the cell differently from its noise-corrected historical experience.

4 Benchmark results

Table 1 and Figure 3 report a run of `python main.py -sample` at seed 42 on the committed sample (hash `b1ba275532b83033`, base rate 0.2212), 3,750 train and 1,250 test rows, 49 predictors. Class weights are left at their natural values throughout: rebalancing would lift ROC-AUC and damage the calibration that the ECE and Brier gates exist to measure, and a probability of default that no longer means “probability of default” is not usable for provisioning.

Table 1: Benchmark and gate verdicts, committed offline sample, seed 42. Discrimination metrics are test-set; CV columns are 5-fold stratified cross-validation on the training split. This run is Windows / Python 3.13.7; `logistic_regression` and `lightgbm` are bit-identical on Linux, `xgboost` is not (Section 6).

Model	ROC-AUC	PR-AUC	KS	Brier	Log-loss	ECE	CV mean	CV s.d.	Verdict
<code>lightgbm</code>	0.7634	0.5355	0.4220	0.1374	0.4469	0.0411	0.7671	0.0179	FAIL
<code>xgboost</code>	0.7625	0.5318	0.4215	0.1379	0.4474	0.0438	0.7653	0.0154	FAIL
<code>logistic_regression</code>	0.7607	0.5042	0.4155	0.1403	0.4459	0.0333	0.7511	0.0224	FAIL

The three models span 0.0026 of ROC-AUC against fold-to-fold standard deviations of 0.0154 to 0.0224. Reading that as a ranking would be overreading it, which is the practical argument for reporting the gate matrix alongside the leaderboard rather than beneath it. Gradient boosting does earn a real margin on PR-AUC — 0.5355 against 0.5042 — so the tree models are better at concentrating the minority class, even though their ordering between themselves is not resolvable at this sample size.

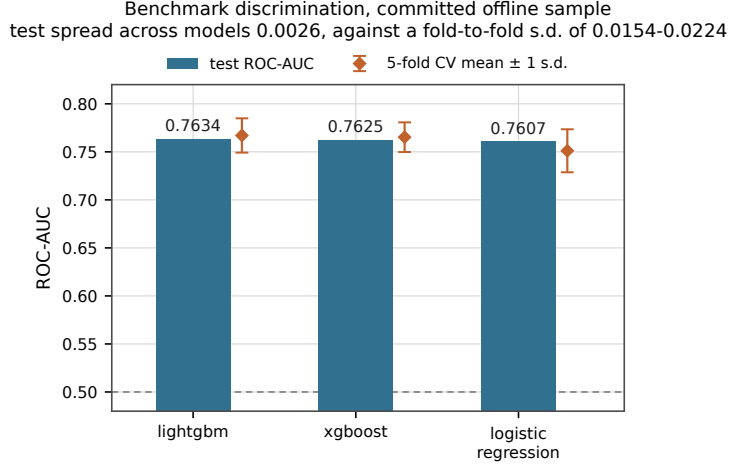


Figure 3: Test ROC-AUC per model with the cross-validation mean and standard deviation alongside; the dashed line marks chance at 0.50. The spread between models is roughly an order of magnitude smaller than the fold-to-fold spread within each model.

5 Governance findings

Table 2 gives the gate matrix. The overall verdict is FAIL for every model, which is the intended outcome rather than a broken build: a governance harness tuned so that everything passes is not measuring anything.

Table 2: Gate verdicts. The worst gate is the model’s verdict.

Gate	lightgbm	xgboost	logistic_regression
fairness	FAIL	FAIL	FAIL
drift	PASS	PASS	PASS
calibration	WARN	WARN	WARN
uncertainty	PASS	PASS	PASS
segment_stability	FAIL	FAIL	WARN

5.1 Fairness: a gap on an attribute the model never saw

The sharpest result is on **sex** under logistic regression. **sex** is withheld from the feature matrix entirely, along with **marriage**, and kept only as an audit column. The model nonetheless produces an equalized-odds gap of **0.1617**, past the 0.10 fail limit, while passing demographic parity at 0.0459. Table 3 shows why: the gap is almost entirely in the true-positive rate, where the false-positive-rate spread is only 0.0084.

Table 3: Fairness detail for **sex**, logistic regression, decision threshold 0.5. Demographic parity gap 0.0459 (pass); equalized-odds gap **0.1617** (fail, limit 0.10).

sex	n	Predicted default rate	Observed default rate	TPR	FPR
male (1)	495	0.1333	0.2283	0.4071	0.0524
female (2)	755	0.0874	0.2159	0.2454	0.0439

Two things follow. First, a parity-only fairness check would have signed this model off: predicted default rates are close across the two groups, and it is recall on borrowers who actually defaulted that is not — a defaulting male applicant is identified 41% of the time against 25% for a defaulting female applicant. Second, and more to the point, the model produced that gap without ever reading the column; the effect arrives through correlated repayment behaviour. That is exactly the case dropping a protected attribute does not cover, and the reason the gate audits withheld attributes rather than treating exclusion as sufficient. Logistic

regression is also one of the two models that reproduce bit-identically across platforms, so this figure is not an artefact of the machine it was computed on.

The framework audits `age_band` as well, and the contrast is instructive rather than equivalent. `age` is used directly as a model feature — 12th of 49 by SHAP importance on LightGBM (Section 7) — and its equalized-odds gap of 0.1600 on LightGBM therefore reflects the model using an input it was handed. That raises the question of whether the use is justified; it is not evidence of a hidden proxy in the way the `sex` result is. The 60–80 band is excluded from the gate entirely: 10 test rows is below the configured `min_group_size` of 50, and reporting a true-positive rate on that is worse than reporting nothing.

5.2 Calibration, drift and uncertainty

Calibration warns on all three models. LightGBM’s expected calibration error is 0.0411 against a warn limit of 0.03, with maximum calibration error 0.2266 over 14 populated bins and a mean predicted PD of 0.2060 against an observed rate of 0.2208 — a slight systematic under-prediction of 0.0148. Drift passes cleanly, as it should on a random split: maximum feature PSI is 0.0156 across all 49 features, on `payment_ratio_mean`, with no feature over the 0.10 warn line. Uncertainty passes with 17.8% of the test set above the 0.60-nat predictive-entropy threshold, and the bootstrap 95% interval for LightGBM’s ROC-AUC is [0.7022, 0.8097] — a width of 0.107, which is worth holding alongside the 0.0026 spread that separates the three models.

5.3 Segment stability

The stability gate fails LightGBM: 4 of 11 gated segments price outside their credible interval, a ratio of 0.364 against a fail limit of 0.30. The clearest breach is `graduate_school | 30-40`, where the model’s mean predicted PD is 0.163 against a shrunk posterior mean of 0.213 with interval [0.187, 0.239], built from 560 reference borrowers. `university | 50-60` breaches in the same direction, 0.187 against [0.202, 0.276]. Both are under-predictions: the model prices default risk for these cells below their noise-corrected historical experience. That is a provisioning concern, and no aggregate metric in Table 1 surfaces it. Logistic regression warns rather than fails here, at 0.273.

6 Cross-platform reproducibility

On the same seed and the same sample hash, across Windows and Linux:

- `logistic_regression` and `lightgbm` reproduce **bit-identically** — every metric, every gate verdict, every credible interval.
- The Bayesian layer reproduces bit-identically everywhere; it is pure numpy/scipy.
- `xgboost` does **not**. Its histogram tree builder reduces floating-point sums in a thread- and platform-dependent order, so the same seed gives ROC-AUC 0.7672 on Linux and 0.7625 on Windows, with ECE 0.0320 against 0.0438.

That 0.0047 spread is larger than the 0.0038 margin between the top two models on Linux, so *which model wins depends on the machine it is run on*: XGBoost on Linux, LightGBM on Windows. Table 1 reports the Windows run for that reason, and the CI table in the repository README reports the Linux run.

Every gate verdict in Table 2 is identical on both platforms, which is the part worth keeping: the approval decision is stable even where the leaderboard is not. A governance verdict that flipped with the host would be useless, whereas a benchmark ranking that flips is merely unreliable. Anyone needing a stable winner as well as a stable verdict should pin the platform or set `models.enabled` to a single model.

7 Explainability

SHAP runs on whichever model the selection metric picks, which on this run is LightGBM. Explainer choice is by model family — exact tree SHAP for the boosters, exact linear SHAP for the scaled logistic pipeline — so neither path uses sampling approximation [5]. Both output on the log-odds scale, where contributions are additive: $\sigma(\text{base} + \sum \text{SHAP})$ recovers the predicted probability exactly. A mean absolute SHAP value is therefore movement in log-odds, not a change in probability of default.

Figure 4 shows the result. Most recent repayment status (`pay_status_1`, 0.359) dominates at more than twice the next feature, followed by `max_delinquency` (0.152), `limit_bal` (0.135) and `delinquent_months` (0.122). Delinquency history and credit limit carrying the model is consistent with the source paper [1]. The pipeline also writes per-feature contribution breakdowns for three individual borrowers — the highest-risk, the lowest-risk, and the most uncertain in the sample — as JSON, in the form an adverse-action review needs.

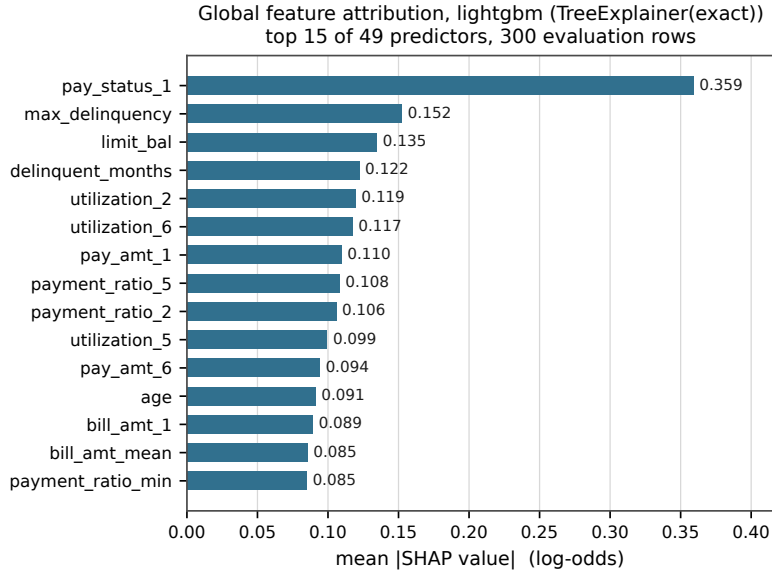


Figure 4: Global feature attribution for LightGBM, exact `TreeExplainer`, top 15 of 49 predictors over 300 evaluation rows against 100 background rows.

8 Dataset, tech stack, and reproducing this report

The data is UCI Machine Learning Repository dataset 350, *Default of Credit Card Clients* [1, 2]: 30,000 Taiwanese credit-card customers, 23 features covering six months of billing and repayment history plus limited demographics, and a binary target for default on the next payment, at a base default rate of 22.1%. It is distributed under CC BY 4.0; the repository commits a 5,000-row class-stratified draw (base rate 22.12%) under the same licence, so the pipeline, its tests, and the CI smoke job run with no network call.

Models are scikit-learn `LogisticRegression`, `xgboost` and `lightgbm`; the Bayesian layer is numpy and scipy only; configuration is pydantic v2 with `extra="forbid"`; explainability is `shap`; tracking is MLflow with a local JSON fallback. CI runs ruff, pytest on Python 3.12 and 3.13, and an end-to-end smoke job that asserts both scorecards and all three model cards were written, that three models were benchmarked with five gates each, that every ROC-AUC beat chance, and that the shrinkage prior was fitted.

Everything in this report regenerates from the tagged release:

```
git clone https://github.com/asynced24/financial-ai-risk-governance-framework.git
cd financial-ai-risk-governance-framework && git checkout v1.0.0
pip install -r requirements.txt
python main.py -sample
python docs/figures/generate_figures.py
```

The first command writes `reports/governance_scorecard.json`, from which Tables 1–3 are taken; the second writes the four figures. Figure 2 re-fits the shrinkage model directly rather than reading a cached artefact. Python 3.12 or newer is required, since the pinned numpy, scipy, xgboost and shap releases all declare `Requires-Python >= 3.12`.

References

- [1] Yeh, I.-C., & Lien, C.-H. (2009). The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications*, 36(2), 2473–2480.
- [2] UCI Machine Learning Repository, dataset 350: *Default of Credit Card Clients*. Distributed under CC BY 4.0. <https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients>
- [3] Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A., & Rubin, D. B. (2013). *Bayesian Data Analysis*, 3rd edition. CRC Press.
- [4] Hardt, M., Price, E., & Srebro, N. (2016). Equality of opportunity in supervised learning. *Advances in Neural Information Processing Systems*, 29.
- [5] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.
- [6] Naeini, M. P., Cooper, G. F., & Hauskrecht, M. (2015). Obtaining well calibrated probabilities using Bayesian binning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 29.